

Integração Python + WEGnology

Monitoramento Simulado de Motor — Pressão, Torque e Temperatura

Relatório de Processo (README)

Este documento descreve o passo a passo completo do processo de integração entre um script Python e a plataforma IoT WEGnology, incluindo o envio de dados simulados de um motor (pressão, torque e temperatura) e a criação de um dashboard de visualização.

1. Objetivo

Testar a integração via Python com o WEGnology, simulando o comportamento de um motor industrial que reporta três variáveis:

- Pressão (bar)
- Torque (N.m)
- Temperatura (°C)

Os dados são enviados via REST API e visualizados em tempo real em um dashboard.

2. Preparação do Ambiente Python

2.1 Instalação da biblioteca

```
pip install wegnology-rest
```

2.2 Problema encontrado: nome do módulo

A documentação oficial do pacote (PyPI) indica o import como `wegnology_rest`, mas o nome real do módulo instalado é **wegnologyrest** (sem underscore). Isso causava `ModuleNotFoundError` mesmo com o pacote instalado corretamente.

Import correto:

```
from wegnologyrest import Client
```

2.3 Problema encontrado: instalação corrompida

Em um primeiro momento, o comando "pip show wegnology-rest" indicava a biblioteca instalada, mas a pasta do pacote não existia fisicamente em site-packages. Resolvido com reinstalação forçada:

```
python -m pip uninstall wegnology-rest -y
python -m pip install --no-cache-dir --force-reinstall wegnology-rest
```

3. Configuração na Plataforma WEGnology

3.1 Criação da Application

- Acessar app.wnology.io
- Criar uma Application isolada e exclusiva para o teste (evitando interferir em aplicações já existentes)
- Template escolhido: Blank Application
- Nome: Teste Motor Python

3.2 Criação do Device

- Dentro da Application → Devices → Add Device
- Device Class: Standalone
- Nome: Motor Teste 01
- Parent System: nenhum
- Device Tags: nenhuma (opcional)

3.3 Criação dos Attributes

Na aba Attributes do device, foram criados três atributos do tipo Number:

Attribute Name	Data Type
pressao	Number
torque	Number
temperatura	Number

Atenção: os nomes dos atributos precisam ser idênticos (case-sensitive) às chaves usadas no JSON enviado pelo Python.

3.4 Criação da Access Key

- Menu da Application → Access Keys → Add Access Key
- Nome: Chave Python Teste

- Device Restrictions: Allow all devices
 - Connection Restrictions: Allow all IP addresses
 - Copiados Key e Secret (o Secret só é exibido uma vez)
-

4. Script Python de Envio de Dados

```
from wegnologyrest import Client
import time
import random

DEVICE_ID = "SEU_DEVICE_ID"
APP_KEY = "SUA_APP_ACCESS_KEY"
APP_SECRET = "SEU_APP_ACCESS_SECRET"

creds = {
    'deviceId': DEVICE_ID,
    'key': APP_KEY,
    'secret': APP_SECRET
}

client = Client()

# Autentica e guarda a resposta (token + applicationId)
auth_response = client.auth.authenticate_device(credentials=creds)
client.auth_token = auth_response['token']
APP_ID = auth_response['applicationId']

print("Autenticado com sucesso!")

def ler_sensores_motor():
    """
    Simula os dados do motor. Pode ser substituído por leitura
    real de sensores (PLC, Modbus, Arduino, etc).
    """
    return {
        "pressao": round(random.uniform(2.0, 8.0), 2),
        "torque": round(random.uniform(50, 200), 2),
        "temperatura": round(random.uniform(60, 110), 2)
    }

def enviar_dados_motor():
    dados = ler_sensores_motor()
    try:
        state = {'data': dados}
        client.device.send_state(
            deviceId=DEVICE_ID,
            applicationId=APP_ID,
            deviceState=state
        )
        print(f"[OK] Enviado -> Pressão: {dados['pressao']} bar | "
              f"Torque: {dados['torque']} N.m | "
              f"Temperatura: {dados['temperatura']} °C")
    except Exception as e:
        print(f"[ERRO] Falha ao enviar: {e}")

if __name__ == "__main__":
    print("Iniciando monitoramento simulado do motor...")
```

```
while True:
    enviar_dados_motor()
    time.sleep(5)
```

4.1 Boas práticas de segurança

- Credenciais (Key/Secret) não devem ser compartilhadas em prints, chats ou repositórios públicos.
- Recomenda-se revogar e recriar a Access Key após testes onde o Secret foi exposto.
- Em produção, usar variáveis de ambiente (.env) em vez de valores fixos no script.

5. Criação do Dashboard

5.1 Estrutura recomendada

- Blocos individuais por variável, cada um com sua própria escala, em vez de um único gráfico com todas as séries misturadas.

Motivo: pressão, torque e temperatura têm unidades e faixas diferentes; combiná-las no mesmo eixo distorce a leitura visual.

5.2 Blocos criados

Bloco	Tipo	Attribute	Escala
Temperatura do Motor	Time Series Graph	temperatura	50–150 °C
Pressão do Motor	Gauge	pressao	0–10 bar
Torque do Motor	Custom HTML (velocímetro)	torque	0–250 N.m

5.3 Configuração do Time Series (Temperatura)

- Data Type: Live Stream
- Duration: 15–30 minutos
- Aggregation: Last
- Line Type: Linear
- Line Wt.: 2 / Dot Wt.: 3
- Y Axis Min/Max: 50 / 150

5.4 Velocímetro customizado (Torque) — Bloco Custom HTML

Custom Head Content:

```
<meta charset="UTF-8">
<script src="https://cdn.jsdelivr.net/npm/canvas-gauges@2.1.7/gauge.min.js"></script>
```

```

<style>
  body {
    padding: 10px;
    background: transparent;
    display: flex;
    justify-content: center;
    align-items: center;
  }
</style>

```

Custom Body Content:

```

<canvas id="gauge-container"></canvas>

<script type="text/javascript">
  var gauge = new RadialGauge({
    renderTo: 'gauge-container',
    width: 240,
    height: 240,
    units: "N.m",
    minValue: 0,
    maxValue: 250,
    majorTicks: [0, 50, 100, 150, 200, 250],
    minorTicks: 5,
    strokeTicks: true,
    colorUnits: "#0055ff",
    colorNeedle: "#0066cc",
    colorNeedleEnd: "#0033aa",
    colorNeedleCircleOuter: "#0055ff",
    colorNeedleCircleInner: "#002288",
    colorBorderOuter: "#0277bd",
    colorBorderMiddle: "#004c8c",
    borders: true,
    highlights: [
      { from: 200, to: 250, color: 'rgba(0, 102, 204, 0.35)' }
    ],
    colorPlate: "#fff",
    borderShadowWidth: 0,
    needleType: "arrow",
    needleWidth: 3,
    animationDuration: 500
  }).draw();

  function renderBlock(input) {
    if (!input || !input.queries) return;
    var dataset = input.queries["Torque"] || Object.values(input.queries)[0];
    if (dataset) {
      var valor = Array.isArray(dataset) ? dataset[0]?.value : dataset.value;
      if (valor !== undefined && valor !== null) {
        gauge.value = parseFloat(valor);
      }
    }
  }

  DashboardBlock.on('change', renderBlock);

```

```
if (DashboardBlock.input) {  
  renderBlock(DashboardBlock.input);  
}  
</script>
```

5.5 Problema encontrado: RadialGauge is not defined

O link inicial do CDN (cdnjs.cloudflare.com/ajax/libs/canvas-gauges/...) não existia. A biblioteca canvas-gauges está hospedada no **jsDelivr**, não no cdnjs. Corrigido usando:

```
<script src="https://cdn.jsdelivr.net/npm/canvas-gauges@2.1.7/gauge.min.js"></script>
```

Como foi diagnosticado: abrindo o Console do navegador (F12) na página do dashboard já publicado, onde o erro "Uncaught ReferenceError: RadialGauge is not defined" apareceu explicitamente.

6. Blocos Avaliados e Descartados

Bloco	Motivo de descarte
Pie Chart	Compara proporções entre grandezas de unidades diferentes (bar, N.m, °C), sem significado analítico real
GPS Heatmap	Exige atributo do tipo GPS (latitude/longitude); o device não possui esse tipo de dado
Gráfico de barras combinado	Mesma limitação do Pie Chart — escalas incompatíveis geram leitura distorcida

7. Lições Aprendidas / Troubleshooting

Problema	Causa	Solução
ModuleNotFoundError: No module named 'wegnology_rest'	Nome real do módulo é wegnologyrest (sem underscore)	Corrigir o import
Pacote "instalado" mas módulo não encontrado	Instalação corrompida no site-packages	Desinstalar e reinstalar com --no-cache-dir --force-reinstall
[ERRO] Falha ao enviar: 'applicationId'	Faltava capturar applicationId da resposta de autenticação	Guardar auth_response['applicationId'] e passar no send_state
Gauge nativo com ponteiro fora da escala	Attribute errado selecionado no bloco (ex: temperatura no lugar de pressão)	Revisar o Attribute configurado em cada bloco
Gráfico combinado poluído/ilegível	Múltiplas séries com escalas muito diferentes no mesmo eixo	Separar em blocos individuais por variável
Velocímetro custom não aparecia	URL do CDN incorreta (canvas-gauges não está no cdnjs)	Usar CDN correto via jsDelivr

8. Estrutura Final do Dashboard

```
[ Gauge Temperatura ]   [ Gauge Pressão ]   [ Velocímetro Torque (Custom HTML) ]  
[           Gráfico de Linha – Temperatura (histórico)           ]
```

9. Próximos Passos Sugeridos

- Substituir a simulação (random.uniform) por leitura real de sensores (PLC, Modbus, Arduino).
- Revogar a Access Key usada durante os testes e gerar uma nova, armazenada em variável de ambiente.
- Ajustar o intervalo de envio (time.sleep) conforme a frequência real de amostragem necessária.
- Adicionar tratamento de reconexão/reautenticação em caso de expiração de token (HTTP 401).
- Criar Workflows no WEGnology para alertas automáticos (ex: torque acima de 200 N.m)

10. Foto do dashboard atualmente

